# Natural Language Processing

**Prof. Ahmed Guessoum**
**The National Higher School of AI**

---

## Chapter 3

# Language Modeling

# Language Modeling

Introduction to N-grams

# Probabilistic Language Models

- Today's goal: assign a probability to a sentence (or to the next word(s))
  - Spell Correction
    - The office is about fifteen **minuets** from my house

Why? P(about fifteen **minutes** from) > P(about fifteen **minuets** from)

  - Speech Recognition
    - P(I saw a van) >> P(eyes awe of an)
  - Augmentative and alternative communication
    - People physically unable to speak or sign: can use eye gaze or other movements to select words from a menu.
  - + Summarization, question-answering, etc., etc.

# Probabilistic Language Modeling

- **Goal**: compute the probability of a sentence or sequence of words:

  $P(W) = P(w_1,w_2,w_3,w_4,w_5...w_n)$

- Related task: probability of an upcoming word:

  P(w5|w1,w2,w3,w4)

- A model that computes either of these:

  P(W) or P(wn|w1,w2…wn-1) is called a **language model**.

  … Better: **the grammar**

  But **language model** or **LM** `

- In this chapter, we cover the simplest way to build a LM: Using N-grams!

# How to compute P(W)

- How to compute this joint probability:
  - P(its, water, is, so, transparent, that)
- Intuition: let's rely on the Chain Rule of Probability

# Reminder: The Chain Rule

- Recall the definition of conditional probabilities

$$\mathbf{p(B|A) = P(A,B)/P(A)}$$

Rewriting it as: $\mathbf{P(A,B) = P(A)P(B|A)}$

- More variables:

$$P(A,B,C,D) = P(A)P(B|A)P(C|A,B)P(D|A,B,C)$$

- The Chain Rule in General

$$P(x_1,x_2,x_3,\ldots,x_n) = P(x_1)P(x_2|x_1)P(x_3|x_1,x_2)\ldots P(x_n|x_1,\ldots,x`)$$

# Chain Rule

- Chaine rule applied to compute joint probability of words in a sentence:

P(its water is so transparent) =

P(its) × P(water|its) × P(is|its water)
× P(so|its water is)
× P(transparent|its water is so)

$$P(w_1 w_2 \ldots w_n) = \prod_i P(w_i \mid w_1 w_2 \ldots w_{i-1})$$

# How to estimate these probabilities

- Could we just count and divide?

$$P(\text{the} \mid \text{its water is so transparent that}) = \frac{Count(\text{its water is so transparent that the})}{Count(\text{its water is so transparent that})}$$

- No! Too many possible sentences!
- Language is creative!
- We'll never see enough data for estimating these ➔ Risk of having probabilities = 0!

# Markov Assumption



Andrei Markov

- **Markov assumption**: *the probability of a word depends only on the previous word*.

- Markov models: the class of probabilistic models that assume we can predict the probability of some future unit without looking too far into the past.

# Markov Assumption



Andrei Markov

- Simplifying assumption (using bigrams):

$$P(\text{the} \mid \text{its water is so transparent that}) \approx P(\text{the} \mid \text{that})$$

- Or maybe (using trigrams):

$$P(\text{the} \mid \text{its water is so transparent that}) \approx P(\text{the} \mid \text{transparent that})$$

- Etc. with N-grams!

# Markov Assumption

$$P(w_1 w_2 \ldots w_n) \approx \prod_i P(w_i \mid w_{i-k} \ldots w_{i-1})$$

- In other words, we approximate each component in the product

$$P(w_i \mid w_1 w_2 \ldots w_{i-1}) \approx P(w_i \mid w_{i-k} \ldots w_{i-1})$$

# Simplest case: Unigram model

$$P(w_1 w_2 \ldots w_n) \approx \prod_i P(w_i)$$

Some automatically generated sentences from a unigram model

- fifth, an, of, futures, the, an, incorporated, a, a, the, inflation, most, dollars, quarter, in, is, mass
- thrift, did, eighty, said, hard, 'm, july, bullish
- that, or, limited, the

# Bigram model

- A condition rests on the previous word:

$$P(w_i \mid w_1 w_2 \ldots w_{i-1}) \approx P(w_i \mid w_{i-1})$$

Some automatically generated sentences from a unigram model:

- texaco, rose, one, in, this, issue, is, pursuing, growth, in, a, boiler, house, said, mr., gurria, mexico, 's, motion, control, proposal, without, permission, from, five, hundred, fifty, five, yen
- outside, new, car, parking, lot, of, the, agreement, reached
- this, would, be, a, record, november

# N-gram models

- We can extend to trigrams, 4-grams, 5-grams
- In general this is an insufficient model of language
  - because language has **long-distance dependencies**, e.g.:

"The computer which I had just put into the machine room on the fifth floor crashed."

- But we can often get away with N-gram models

# Language Modeling

## Estimating N-gram Probabilities

# Estimating bigram probabilities

- The Maximum Likelihood Estimate

$$P(w_i \mid w_{i-1}) = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

$$P(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

# Example

$$P(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and lham </s>

$$P(\text{I} \mid \text{<s>}) = \frac{2}{3} = .67 \qquad P(\text{Sam} \mid \text{<s>}) = \frac{1}{3} = .33 \qquad P(\text{am} \mid \text{I}) = \frac{2}{3} = .67$$

$$P(\text{</s>} \mid \text{Sam}) = \frac{1}{2} = 0.5 \qquad P(\text{Sam} \mid \text{am}) = \frac{1}{2} = .5 \qquad P(\text{do} \mid \text{I}) = \frac{1}{3} = .33$$

# More examples: Berkeley Restaurant Project sentences

- can you tell me about any good cantonese restaurants close by
- mid priced thai food is what i'm looking for
- tell me about chez panisse
- can you give me a listing of the kinds of food that are available
- i'm looking for a good place to eat breakfast
- when is caffe venezia open during the day

# Raw bigram counts

- Out of 9222 sentences…

| | i | want | to | eat | chinese | food | lunch | spend |
|---|---|---|---|---|---|---|---|---|
| i | 5 | 827 | 0 | 9 | 0 | 0 | 0 | 2 |
| want | 2 | 0 | 608 | 1 | 6 | 6 | 5 | 1 |
| to | 2 | 0 | 4 | 686 | 2 | 0 | 6 | 211 |
| eat | 0 | 0 | 2 | 0 | 16 | 2 | 42 | 0 |
| chinese | 1 | 0 | 0 | 0 | 0 | 82 | 1 | 0 |
| food | 15 | 0 | 15 | 0 | 1 | 4 | 0 | 0 |
| lunch | 2 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| spend | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |

# Raw bigram probabilities

- Normalize by unigrams:

| i | want | to | eat | chinese | food | lunch | spend |
|---|---|---|---|---|---|---|---|
| 2533 | 927 | 2417 | 746 | 158 | 1093 | 341 | 278 |

- Result:

| | i | want | to | eat | chinese | food | lunch | spend |
|---|---|---|---|---|---|---|---|---|
| i | 0.002 | 0.33 | 0 | 0.0036 | 0 | 0 | 0 | 0.00079 |
| want | 0.0022 | 0 | 0.66 | 0.0011 | 0.0065 | 0.0065 | 0.0054 | 0.0011 |
| to | 0.00083 | 0 | 0.0017 | 0.28 | 0.00083 | 0 | 0.0025 | 0.087 |
| eat | 0 | 0 | 0.0027 | 0 | 0.021 | 0.0027 | 0.056 | 0 |
| chinese | 0.0063 | 0 | 0 | 0 | 0 | 0.52 | 0.0063 | 0 |
| food | 0.014 | 0 | 0.014 | 0 | 0.00092 | 0.0037 | 0 | 0 |
| lunch | 0.0059 | 0 | 0 | 0 | 0 | 0.0029 | 0 | 0 |
| spend | 0.0036 | 0 | 0.0036 | 0 | 0 | 0 | 0 | 0 |

# Bigram estimates of sentence probabilities

P(<s> I want english food </s>) =

P(I|<s>)

× P(want|I)

× P(english|want)

× P(food|english)

× P(</s>|food)

= .000031

# What kinds of knowledge?

- P(english|want) = .0011
- P(chinese|want) = .0065
- P(to|want) = .66
- P(eat | to) = .28
- P(food | to) = 0
- P(want | spend) = 0
- P (i | <s>) = .25

World knowledge

Syntax

Discourse

# Practical Issues

- Multiplying (small) probabilities may lead to 0 (even division by 0)!
- Do everything in log space
  - Mainly to avoid underflow
  - (also adding is faster than multiplying)

$$\log(p_1 * p_2 * p_3 * p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

# Language Modeling Toolkits

- Efficient language model toolkits like KenLM use sorted arrays and use merge sorts to efficiently build the proba. tables in a minimal number of passes through a large corpus.
- KenLM
  - https://kheafield.com/code/kenlm/
- Another LM Toolkit: SRILM
  - http://www.speech.sri.com/projects/srilm/

# Google N-Gram Release, August 2006

AUG
3

## All Our N-gram are Belong to You

Posted by Alex Franz and Thorsten Brants, Google Machine Translation Team

Here at Google Research we have been using word n-gram models for a variety of R&D projects,

...

That's why we decided to share this enormous dataset with everyone. We processed 1,024,908,267,229 words of running text and are publishing the counts for all 1,176,470,663 five-word sequences that appear at least 40 times. There are 13,588,391 unique words, after discarding words that appear less than 200 times.

# Google N-Gram Release

- serve as the incoming 92
- serve as the incubator 99
- serve as the independent 794
- serve as the index 223
- serve as the indication 72
- serve as the indicator 120
- serve as the indicators 45
- serve as the indispensable 111
- serve as the indispensible 40
- serve as the individual 234

http://googleresearch.blogspot.com/2006/08/all-our-n-gram-are-belong-to-you.html

# Language Modeling

## Evaluation and Perplexity

# How to evaluate N-gram models

- One way to evaluate a model, embed it in an application and measure how much the application improves: **Extrinsic Evaluation**.
- Extrinsic (in-vivo) Evaluation: To compare models A and B
  1. Put each model in a real task
     - Machine Translation, speech recognition, etc.
  2. Run the task, get a score for A and for B
     - How many words translated correctly
     - How many words transcribed correctly
     - …
  3. Compare accuracy for A and B

# Intrinsic (in-vitro) evaluation

- Extrinsic evaluation not always possible
  - Expensive, time-consuming
- **Intrinsic evaluation metric:** one that measures the quality of a model independent of any application.
- **Perplexity:** standard intrinsic metric for measuring LM performance:
  - Directly measures LM performance at predicting words.
  - Doesn't necessarily correspond with real application performance
  - Gives us a single general metric for LMs
  - Useful for large language models (LLMs) as well as n-grams

# Training sets and test sets

- We train parameters of our model on a **training set**.
- We test the model's performance on data we haven't seen.
  - A **test set** i.e. unseen dataset; different from training set.
    - Intuition: we want to measure generalization to unseen data.
- An evaluation metric (like perplexity) tells us how well our model does on the test set.

# Choosing training and test sets

- If we're building an LM for a specific task
  - The test set should reflect the task language we want to use the model for.
- If we're building a general-purpose model
  - We'll need lots of different kinds of training data.
  - We don't want the training set or the test set to be just from one domain or author or language.

# Training on the test set

- We can't allow test sentences into the training set.
  - Or else the LM will assign that sentence an artificially high probability when we see it in the test set.
  - And hence assign the whole test set a falsely high probability.
  - Making the LM look better than it really is.
- This is called "Training on the test set"
- Bad science!

# Dev sets

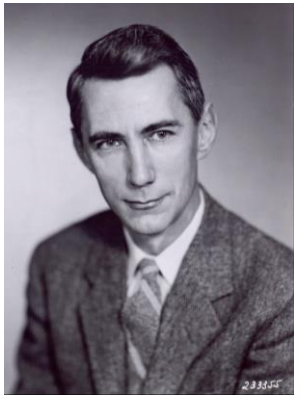
- If we test on the test set many times we might implicitly tune to its characteristics
  - Noticing which changes make the model better.
- So we run on the test set only once, or a few times.
- That means we need a third dataset:
  - A **development test set** or, **devset** (also called: **Validation set)**.
  - We test our LM on the devset until the very end
  - And then test our LM on the **test set** once

# Intuition of Perplexity: Predicting Upcoming Words

The Shannon Game: **How well can we predict the next word**?

- Once upon a _____
- That is a picture of a _____
- For breakfast I ate my usual _____

| | |
|---|---|
| **time** | **0.9** |
| **dream** | **0.03** |
| **midnight** | **0.02** |
| **…** | |
| **and** | **1e-100** |



Claude Shannon

Unigrams are terrible at this game (Why?)

A good LM is one that assigns a higher probability to the next word that actually occurs

Picture credit: Historiska bildsamlingen
https://creativecommons.org/licenses/by/2.0/

# Intuition of perplexity

- We said: a good LM is one that assigns a higher probability to the next word that actually occurs.
- Generalizing this to all the words:
  - The best LM assigns high probability to (i.e. best predicts) the entire test set.
- When comparing two LMs, A and B
  - We compute $P_A$(test set) and $P_B$(test set)
  - The better LM will give a higher probability to (=be less surprised by) the test set than the other LM.

# Intuition of perplexity

- Use perplexity instead of raw probability
- Probability depends on the size of the test set
  - Probability gets smaller the longer the text
  - Better to have a metric that is **per-word**, normalized by length. ➔ we could compare across texts of different lengths.
- **Perplexity** is the inverse probability of the test set, normalized by the number of words

$$PP(W) = P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}} = \sqrt[N]{\frac{1}{P(w_1 w_2 \ldots w_N)}}$$

# Intuition of perplexity

- Perplexity is one of the most important metrics in NLP, used for evaluating large language models as well as n-gram models.
- **Perplexity** is the **inverse** probability of the test set, normalized by the number of words (or tokens).

$$PP(W) = P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}} = \sqrt[N]{\frac{1}{P(w_1 w_2 \ldots w_N)}}$$

- Probability range is [0,1], perplexity range is [1,∞]
- **The higher the probability of the word sequence, the lower the perplexity.** Minimizing perplexity amounts to maximizing the test set probability

# Intuition of perplexity: N-grams

$$PP(W) = P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}}$$
$$= \sqrt[N]{\frac{1}{P(w_1 w_2 \ldots w_N)}}$$

Chain rule: $$\mathrm{PP}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_1 \ldots w_{i-1})}}$$

Bigrams: $$\mathrm{PP}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_{i-1})}}$$

# Intuition of perplexity: Weighted average branching factor

- Perplexity is also the **weighted average branching factor** of a language.
- **Branching factor**: number of possible next words that can follow any word

Example: Deterministic language L = {red, blue, green}

Branching factor = 3 (any word can be followed by red, blue, green)

Now assume LM A where each word follows any other word with equal probability ⅓

Given a test set T = "red red red red blue"

$Perplexity_A(T) = P_A(\text{red red red red blue}) = ((⅓)^5)^{-1/5} = (⅓)^{-1} = 3$

- But now suppose red was very likely in training set, such that for LM B:
  - P(red) = .8  p(green) = .1  p(blue) = .1
- We would expect the probability to be higher, and hence the perplexity to be smaller:

$Perplexity_B(T) = P_B(\text{red red red red blue})^{-1/5} = (.8 * .8 * .8 * .8 * .1)^{-1/5}$

$= .04096^{-1/5} = .527^{-1} = 1.89$

# Lower perplexity → better language model

- Given a text W, different language models will have different perplexities.
- Training set 38 million words, test set 1.5 million words, of WSJ

| N-gram Order | Unigram | Bigram | Trigram |
|---|---|---|---|
| Perplexity | 962 | 170 | 109 |

# Language Modeling

## Sampling and Generalization

- **Sampling from a language:** important way to visualize what kind of knowledge it embodies.
- Sampling from a distribution: choose random points according to their likelihood.
- Sampling from a LM: generate sentence words according to their likelihoods as defined by the LM.
- So we are more likely to generate sentences that the model thinks have a high probability and less likely to generate sentences that the model thinks have a low probability.

# The Shannon (1948) Visualization Method



Claude Shannon

- Some sentences generated by Claude Shannon in 1948 from a unigram and a bigram model.

- Unigram:

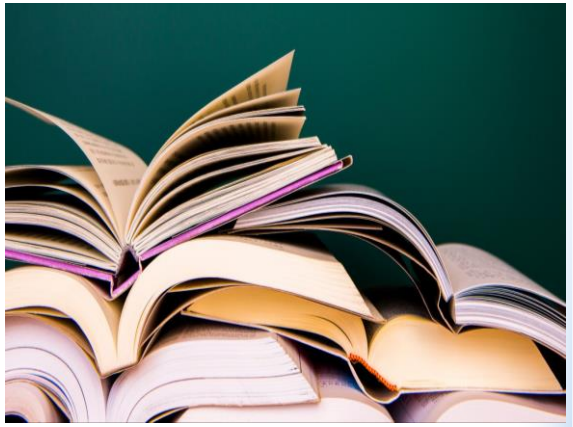
REPRESENTING AND SPEEDILY IS AN GOOD APT OR COME CAN DIFFERENT NATURAL HERE HE THE A IN CAME THE TO OF TO EXPERT GRAY COME TO FURNISHES THE LINE MESSAGE HAD BE THESE.

- Bigram:

THE HEAD AND IN FRONTAL ATTACK ON AN ENGLISH WRITER THAT THE CHARACTER OF THIS POINT IS THEREFORE ANOTHER METHOD FOR THE LETTERS THAT THE TIME OF WHO EVER TOLD THE PROBLEM FOR AN UNEXPECTED.

# How Shannon sampled those words in 1948

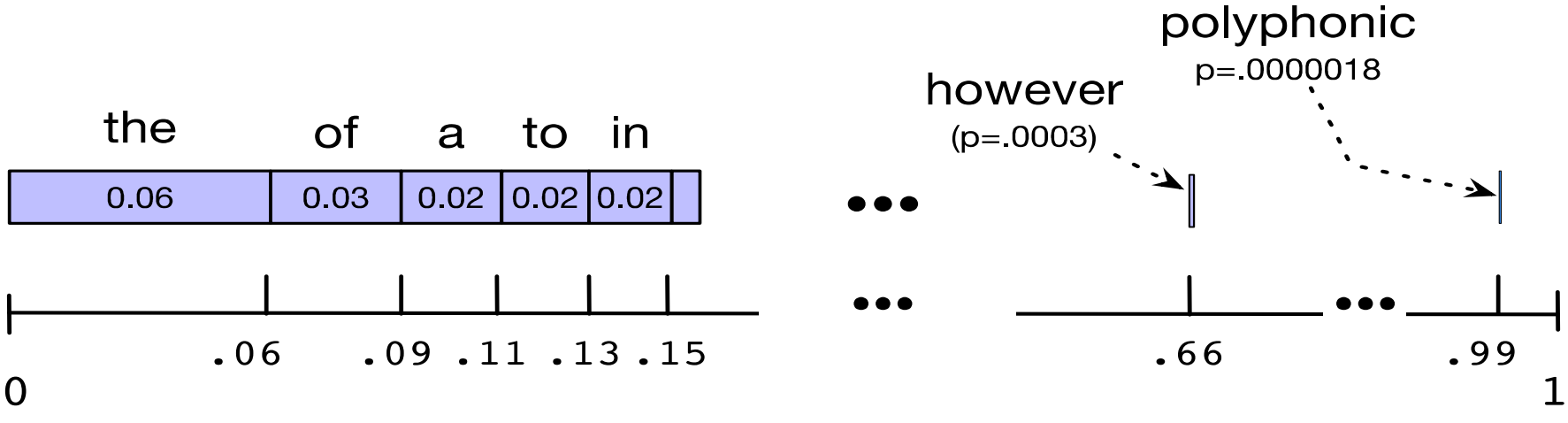
"Open a book at random and select a letter at random on the page. This letter is recorded. The book is then opened to another page and one reads until this letter is encountered. The succeeding letter is then recorded. Turning to another page this second letter is searched for and the succeeding letter recorded, etc."



# Sampling a word from a distribution

Imagine all the words of the English language covering the number line between 0 and 1, each word covering an interval proportional to its frequency.

- Choose a random value between 0 and 1,
- Find that point on the (cumulative) probability line, and
- Print the word whose interval includes this chosen value.
- Continue choosing random numbers and generating words until the sentence-final token </s> is randomly generated.



# Visualizing Bigrams the Shannon Way

- Choose a random bigram (<s>, w) according to its (bigram) probability p(w|<s>)
- Now choose a random bigram (w, x) according to its probability p(x|w)
- And so on until we choose </s>
- Then string the words together

```
<s> I
    I want
      want to
           to eat
              eat Chinese
                  Chinese food
                          food  </s>
I want to eat Chinese food
```

# Other sampling methods

- Other sampling methods are used for neural LMs.
- Many of them avoid generating words from the very unlikely tail of the distribution
- Will be discussed when we get to neural LM decoding (See in Chapter 10: Sampling for LLM Generation):
  - Temperature sampling
  - Top-k sampling
  - Top-p sampling

# Approximating Shakespeare

| | |
|---|---|
| 1 gram | –To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have |
| | –Hill he late speaks; or! a more to leg less first you enter |
| 2 gram | –Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow. |
| | –What means, sir. I confess she? then all sorts, he is trim, captain. |
| 3 gram | –Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done. |
| | –This shall forbid it should be branded, if renown made it empty. |
| 4 gram | –King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in; |
| | –It cannot be but so. |

# Shakespeare as corpus

- N = 884,647 tokens, V = 29,066
- Shakespeare produced 300,000 bigram types out of $V^2$= 844 million possible bigrams.
  - So 99.96% of the possible bigrams were never seen (have zero entries in the table)
  - This is the biggest problem in language modeling; we'll come back to this "Sparsity" issue.
  - That sparsity is even worse for 4-grams, explaining why our sampling generated actual Shakespeare.

# The Wall Street Journal is not Shakespeare

1 gram

Months the my and issue of year foreign new exchange's september were recession exchange new endorsed a acquire to six executives

2 gram

Last December through the way to preserve the Hudson corporation N. B. E. C. Taylor would seem to complete the major central planners one point five percent of U. S. E. has already old M. X. corporation of living on information such as more frequently fishing to keep her

3 gram

They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and Brazil on market conditions

# Can you guess the author?

These 3-gram sentences are sampled from an LM trained on who (WSJ, Shakespeare, or Jane Austen)?

1) They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and gram Brazil on market conditions

2) This shall forbid it should be branded, if renown made it empty.

3) “You are uniformly charming!” cried he, with a smile of associating and now and then I bowed and they perceived a chaise and four to wish for.

# Choosing training data

- If task-specific, use a training corpus that has a similar genre to your task.
  - If legal or medical, need lots of special-purpose documents
- Make sure to cover different kinds of dialects and speaker/authors.
  - Example: *African-American Vernacular English (AAVE)*
    - One of many varieties that can be used by African Americans and others
    - Can include the auxiliary verb **finna** that marks immediate future tense:

      "My phone finna die"

# The Perils of Overfitting

- Problem with using maximum likelihood estimates for probabilities: any finite training corpus will be missing some perfectly acceptable English word sequences.
- N-grams only work well for word prediction if the test corpus looks like the training corpus
  - But even when we try to pick a good training corpus, the test set will surprise us!
  - We need to train robust models that generalize!
- One kind of generalization: **Zeros**
    - Things that don't ever occur in the training set
      - But occur in the test set

# Zeros

- Training set:
  … ate lunch
  … ate dinner
  … ate a
  … ate the

  P(“breakfast” | ate) = 0

- Test set
  … ate lunch
  … ate breakfast

# Zero probability bigrams

- Bigrams with zero probability
  - Will hurt our performance for texts where those words appear!
  - And mean that we will assign 0 probability to the test set!
- And hence we cannot compute perplexity (can't divide by 0)!
- Standard way to deal with "zero probability n-grams" that should really have some non-zero probability is called **smoothing**

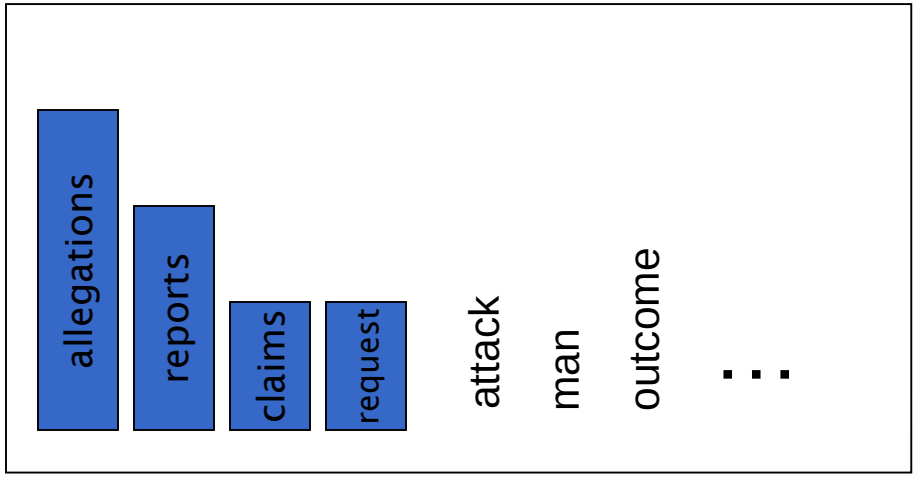
# Language Modeling

Smoothing: Add-one (Laplace) smoothing

# The intuition of smoothing

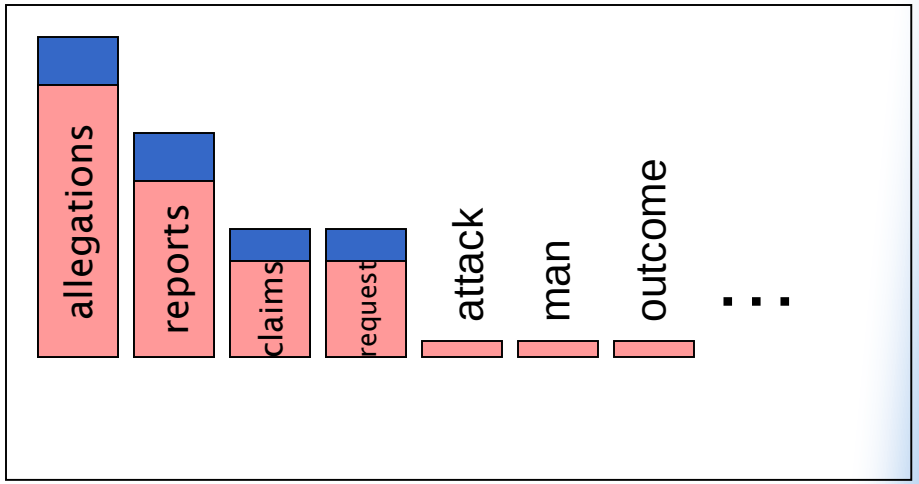
- When we have sparse statistics:

  P(w | denied the)
  3 allegations
  2 reports
  1 claims
  1 request
  **7 total**




- Steal probability mass to generalize better

  P(w | denied the)
  2.5 allegations
  1.5 reports
  0.5 claims
  0.5 request
  2 other
  **7 total**



# Add-one estimation

- Also called Laplace smoothing
- Pretend we saw each word one more time than we did
- Just add one to all the counts!
- MLE estimate: $$P_{MLE}(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$
- Add-1 estimate: $$P_{Add-1}(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i)+1}{c(w_{i-1})+V}$$

# Maximum Likelihood Estimates

- The maximum likelihood estimate
  - of some parameter of a model M from a training set T
  - maximizes the likelihood of the training set T given the model M
- Suppose the word "bagel" occurs 400 times in a corpus of a million words
- What is the probability that a random word from some other text will be "bagel"?
- MLE estimate is 400/1,000,000 = .0004
- This may be a bad estimate for some other corpus
  - But it is the **estimate** that makes it **most likely** that "bagel" will occur 400 times in a million word corpus.

# Berkeley Restaurant Corpus: Laplace smoothed bigram counts

| | i | want | to | eat | chinese | food | lunch | spend |
|---|---|---|---|---|---|---|---|---|
| i | 6 | 828 | 1 | 10 | 1 | 1 | 1 | 3 |
| want | 3 | 1 | 609 | 2 | 7 | 7 | 6 | 2 |
| to | 3 | 1 | 5 | 687 | 3 | 1 | 7 | 212 |
| eat | 1 | 1 | 3 | 1 | 17 | 3 | 43 | 1 |
| chinese | 2 | 1 | 1 | 1 | 1 | 83 | 2 | 1 |
| food | 16 | 1 | 16 | 1 | 2 | 5 | 1 | 1 |
| lunch | 3 | 1 | 1 | 1 | 1 | 2 | 1 | 1 |
| spend | 2 | 1 | 2 | 1 | 1 | 1 | 1 | 1 |

# Laplace-smoothed bigrams

$$P^*(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)+1}{C(w_{n-1})+V}$$

| | i | want | to | eat | chinese | food | lunch | spend |
|---|---|---|---|---|---|---|---|---|
| i | 0.0015 | 0.21 | 0.00025 | 0.0025 | 0.00025 | 0.00025 | 0.00025 | 0.00075 |
| want | 0.0013 | 0.00042 | 0.26 | 0.00084 | 0.0029 | 0.0029 | 0.0025 | 0.00084 |
| to | 0.00078 | 0.00026 | 0.0013 | 0.18 | 0.00078 | 0.00026 | 0.0018 | 0.055 |
| eat | 0.00046 | 0.00046 | 0.0014 | 0.00046 | 0.0078 | 0.0014 | 0.02 | 0.00046 |
| chinese | 0.0012 | 0.00062 | 0.00062 | 0.00062 | 0.00062 | 0.052 | 0.0012 | 0.00062 |
| food | 0.0063 | 0.00039 | 0.0063 | 0.00039 | 0.00079 | 0.002 | 0.00039 | 0.00039 |
| lunch | 0.0017 | 0.00056 | 0.00056 | 0.00056 | 0.00056 | 0.0011 | 0.00056 | 0.00056 |
| spend | 0.0012 | 0.00058 | 0.0012 | 0.00058 | 0.00058 | 0.00058 | 0.00058 | 0.00058 |

# Reconstituted counts

$$c^*(w_{n-1}w_n) = \frac{[C(w_{n-1}w_n)+1] \times C(w_{n-1})}{C(w_{n-1})+V}$$

| | i | want | to | eat | chinese | food | lunch | spend |
|---|---|---|---|---|---|---|---|---|
| i | 3.8 | 527 | 0.64 | 6.4 | 0.64 | 0.64 | 0.64 | 1.9 |
| want | 1.2 | 0.39 | 238 | 0.78 | 2.7 | 2.7 | 2.3 | 0.78 |
| to | 1.9 | 0.63 | 3.1 | 430 | 1.9 | 0.63 | 4.4 | 133 |
| eat | 0.34 | 0.34 | 1 | 0.34 | 5.8 | 1 | 15 | 0.34 |
| chinese | 0.2 | 0.098 | 0.098 | 0.098 | 0.098 | 8.2 | 0.2 | 0.098 |
| food | 6.9 | 0.43 | 6.9 | 0.43 | 0.86 | 2.2 | 0.43 | 0.43 |
| lunch | 0.57 | 0.19 | 0.19 | 0.19 | 0.19 | 0.38 | 0.19 | 0.19 |
| spend | 0.32 | 0.16 | 0.32 | 0.16 | 0.16 | 0.16 | 0.16 | 0.16 |

# Compare with raw bigram counts

| | i | want | to | eat | chinese | food | lunch | spend |
|---|---|---|---|---|---|---|---|---|
| i | 5 | 827 | 0 | 9 | 0 | 0 | 0 | 2 |
| want | 2 | 0 | 608 | 1 | 6 | 6 | 5 | 1 |
| to | 2 | 0 | 4 | 686 | 2 | 0 | 6 | 211 |
| eat | 0 | 0 | 2 | 0 | 16 | 2 | 42 | 0 |
| chinese | 1 | 0 | 0 | 0 | 0 | 82 | 1 | 0 |
| food | 15 | 0 | 15 | 0 | 1 | 4 | 0 | 0 |
| lunch | 2 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| spend | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |

| | i | want | to | eat | chinese | food | lunch | spend |
|---|---|---|---|---|---|---|---|---|
| i | 3.8 | 527 | 0.64 | 6.4 | 0.64 | 0.64 | 0.64 | 1.9 |
| want | 1.2 | 0.39 | 238 | 0.78 | 2.7 | 2.7 | 2.3 | 0.78 |
| to | 1.9 | 0.63 | 3.1 | 430 | 1.9 | 0.63 | 4.4 | 133 |
| eat | 0.34 | 0.34 | 1 | 0.34 | 5.8 | 1 | 15 | 0.34 |
| chinese | 0.2 | 0.098 | 0.098 | 0.098 | 0.098 | 8.2 | 0.2 | 0.098 |
| food | 6.9 | 0.43 | 6.9 | 0.43 | 0.86 | 2.2 | 0.43 | 0.43 |
| lunch | 0.57 | 0.19 | 0.19 | 0.19 | 0.19 | 0.38 | 0.19 | 0.19 |
| spend | 0.32 | 0.16 | 0.32 | 0.16 | 0.16 | 0.16 | 0.16 | 0.16 |

# Impact of Add-1 Smoothing

- Add-one smoothing has made a very big change to the counts.
  - E.g. C(want to) changed from 608 to 238!
- Can also be seen in probability space: P(to|want) decreases from 0.66 in the unsmoothed case to 0.26 in the smoothed case.
- In terms of **discount d** (ratio between new and old counts): discount for the bigram "want to" is 0.39 (238/608), discount for "to spend" 0.63 (133/211), while discount for "Chinese food" is 0.987 (82/83)!

# Add-1 estimation is a blunt instrument

- So add-1 isn't used for N-grams:
  - We'll see better methods
- But add-1 is used to smooth other NLP models
  - For text classification
  - In domains where the number of zeros isn't so huge.

# Language Modeling

Interpolation, Backoff, and Web-Scale LMs

# Backoff and Interpolation

- Sometimes it helps to use **less** context
  - Condition on less context for contexts you haven't learned much about
- **Backoff** (use of n-gram hierarchy)**:**
  - Use trigram if you have good evidence,
  - Otherwise bigram, otherwise unigram
- **Interpolation:**
  - Mix unigram, bigram, trigram
  - Weighting and combining the n-grams
- Interpolation works better

# Linear Interpolation

- Simple interpolation

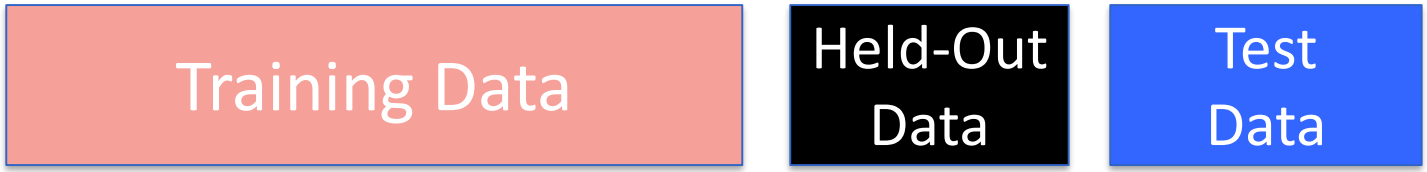
$$\begin{aligned}\hat{P}(w_n|w_{n-2}w_{n-1}) &= \lambda_1 P(w_n|w_{n-2}w_{n-1}) \\ &\quad +\lambda_2 P(w_n|w_{n-1}) \\ &\quad +\lambda_3 P(w_n)\end{aligned} \qquad \sum_i \lambda_i = 1$$

- Lambdas conditional on context:

$$\begin{aligned}\hat{P}(w_n|w_{n-2}w_{n-1}) &= \lambda_1(w_{n-2}^{n-1})P(w_n|w_{n-2}w_{n-1}) \\ &\quad +\lambda_2(w_{n-2}^{n-1})P(w_n|w_{n-1}) \\ &\quad +\lambda_3(w_{n-2}^{n-1})P(w_n)\end{aligned}$$

# How to set the lambdas?

- Use a **held-out** corpus




- Choose λs to maximize the probability of held-out data:
  - Fix the N-gram probabilities (on the training data)
  - Then search for λs that give largest probability to held-out set:

$$\log P(w_1 w_2 \ldots w_n | M(\lambda_1 \lambda_2 \ldots \lambda_k) = \sum_i \log P_{M(\lambda_1 \lambda_2 \ldots \lambda_k)}(w_i | w_{i-1})$$

- Various ways to find the optimal set of λs (e.g. EM algorithm, an iterative learning algorithm)

# EM Algorithm

- With n-gram models, the typical approach involves the following:

**1. With the Training Set: Compute N-gram Models (Model building)**: You compute the unigram, bigram, and trigram probabilities based on the training set. This involves counting the occurrences of each n-gram and calculating the corresponding probabilities.

**2. With the Held-out Set: EM Steps**: EM algorithm applied to this set to optimize the parameters (mixture weights) for the model.

- **Initialize Parameters**: Set initial mixture weights for combining the models.
- **E-step**: Using the held-out set, calculate the expected contributions from each n-gram model to the observed data.
- **M-step**: Update the mixture weights based on these expected contributions.
- **Iterate**: Repeat the E-step and M-step until convergence, using the held-out set for evaluation.

- **With a held-out Set:**
  - Avoid Overfitting.
  - Model performance evaluation after each iteration of the EM algorithm, ensuring that the adjustments made lead to improved performance on unseen data.

# The EM Algorithm

**Model Overview**

- **Unigram:** Probability of w3 based solely on its frequency in the corpus, P(w3).
- **Bigram:** Probability of w3 given w2, P(w3|w2).
- **Trigram:** Probability of w3 given w1 and w2, P(w3|w1,w2).
- overall probability of w3 can be expressed as:

$$P(w3|w1,w2) = \lambda1\ P(w3) + \lambda2\ P(w3|w2) + \lambda3\ P(w3|w1,w2)$$

where λ1,λ2,λ3 are the weights (mixture coefficients) that determine the contribution of each model.

**Step 1: Initialize Parameters**

- **Initial Weights:** Set initial values for λ1,λ2,λ3. As equal or based on some heuristic (e.g., prior knowledge about the importance of the n-grams) λ1+λ2+λ3=1.
- **Model Parameters:** Also initialize the counts needed for unigram, bigram, and trigram probabilities from the training corpus.

# The EM Algorithm (cont.)

- **Step 2: E-Step (Expectation Step)**

Calculate the expected counts for each word based on the current estimates of $\lambda 1$, $\lambda 2$, and $\lambda 3$.

**Expected Counts for Trigram:** For a trigram(w1,w2,w3):
$E[C(w3|w1,w2)] = \lambda 3 \; P(w3|w1,w2) \cdot count(w1,w2)$

**Expected Counts for Bigram:** For a bigram (w2,w3): $E[C(w3|w2)] = \lambda 2 \; P(w3|w2) \cdot count(w2)$

**Expected Counts for Unigram:** For a unigram w3:
$E[C(w3)] = \lambda 1 \; P(w3) \cdot N$
(where N is the total number of words)

# The EM Algorithm (cont.)

- **Step 3: M-Step (Maximization Step)**

Update the mixing weights based on the expected counts calculated in the E-step.

**Calculate Total Counts:** For each $w_3$:

$$\text{TotalCount}(w_3) = E[C(w_3)] + E[C(w_3|w_2)] + E[C(w_3|w_1, w_2)]$$

**Update the λ Values:** The new mixing weights are calculated as:

$$\lambda_1' = \sum_{w3} E[C(w_3)] \;/\; \sum_{w3} \text{Total Count}(w_3)$$

$$\lambda_2' = \sum_{w3} E[C(w_3|w_2)] \;/\; \sum_{w3} \text{Total Count}(w_3)$$

$$\lambda_3' = \sum_{w3} E[C(w_3|w_1, w_2)] \sum_{w3} \text{Total Count}(w_3)$$

**Normalization:** Ensure that $\lambda_1' + \lambda_2' + \lambda_3' = 1$

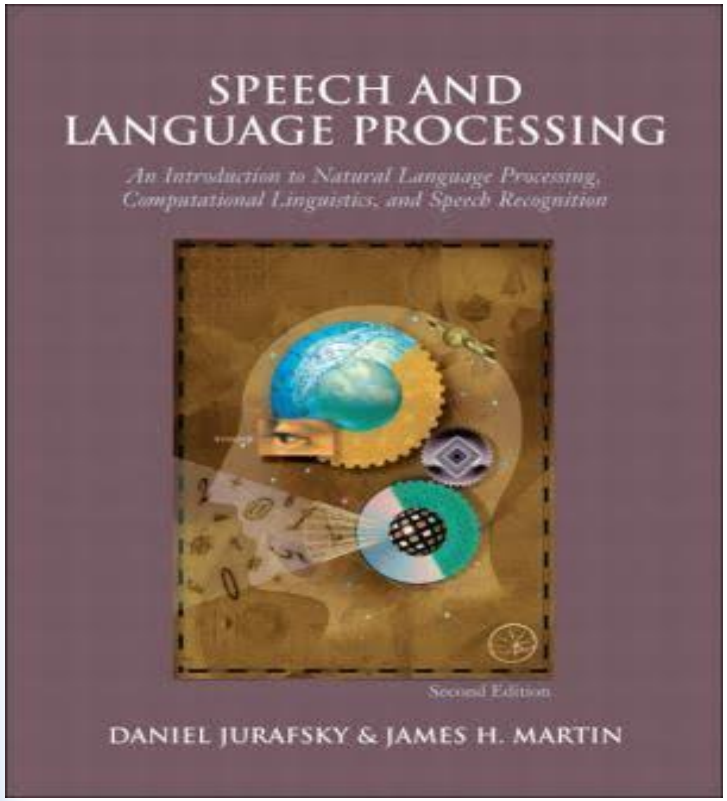
**Step 4: Iterate:** Repeat the E-step and M-step until convergence (i.e., until the changes in $\lambda_1$, $\lambda_2$, and $\lambda_3$ are negligible).

# Unknown words: Open versus closed vocabulary tasks

- If we know all the words in advance
  - Vocabulary V is fixed
  - Closed vocabulary task
- Often we don't know this
  - **Out Of Vocabulary** = OOV words
  - Open vocabulary task
- Instead: create an unknown word token <UNK>
  - Training of <UNK> probabilities
    - Create a fixed lexicon L of size V
    - At text normalization phase, any training word not in L changed to <UNK>
    - Now we train its probabilities like a normal word
  - At decoding time
    - If text input: Use UNK probabilities for any word not in training

# Textbook

Jurafsky, D. and Martin, J.H. (2024) *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models* (3rd ed. draft), Prentice Hall.
https://web.stanford.edu/~jurafsky/slp3/ed3bookaug20_2024.pdf




The draft version of the 3rd edition will be used; it has been updated to include more recent topics such as Vector Semantics and Embeddings, ANNs and Deep Learning for NLP, Transformers, etc.

This course **slides** are largely based on this textbook and its authors' slides with light customisations whenever seen fit.